

Thinking-Trace Analysis: Methodology for Surfacing Operational Improvements and Behavioral-Contract Candidates from Agent Planning Text

Sophie D. Neilson
Independent Researcher
ORCID: [0009-0001-2430-1743](https://orcid.org/0009-0001-2430-1743)

March 2026

Abstract

This paper records a methodology for analyzing AI agent thinking traces: the extended planning text an agent produces before and during task execution. The methodology uses a two-pass structure. The first pass is a piece-by-piece temporal pass that extracts operational improvements, behavioral-contract candidates, vocabulary coinages, and efficiency patterns per trace segment. The second is a structural synthesis pass that ranks improvements by leverage and attributes overhead to either the system (form and protocol gaps) or the agent (judgment patterns). The central analytical question is whether observed overhead is structural or behavioral, because the distinction determines the appropriate remediation. The methodology is demonstrated on one trace from an experimental inquest design task. Limitations include single-trace calibration, analyst-dependent segmentation, and invisibility of absent reasoning.

1 The Analytical Problem

Agent thinking traces are a record of planning that disappears from the accessible context once execution begins. The documents agents produce (journals, inquests, accounts) record outputs: decisions made, artifacts written, acceptance criteria evaluated. They do not record the planning overhead that preceded those outputs. They do not show how many times a direction was taken and abandoned, which constraints were discovered reactively versus anticipated from the form, or where reasoning time was spent building structure the form should have provided.

This overhead is recoverable only from thinking text. Reading a completed artifact tells you what the agent decided. Reading the thinking trace tells you the cost of that decision. It also tells you whether the cost was structural (paid because a form or protocol lacked a gate) or behavioral (paid because the agent's own approach had a pattern worth correcting).

That distinction is the central analytical question. It determines what kind of fix is warranted. Structural overhead calls for form changes: a pre-design gate, mandatory fields, an explicit re-entry protocol. Behavioral overhead calls for agent advice: do X before Y, separate design from execution, read the campaign document first. Conflating the two produces the wrong remediation. Form changes get applied to things only the agent can catch. Agent advice gets applied to things the form should simply require.

Thinking-trace analysis is the instrument for making this distinction reliably.

1.1 Terminology

The methodology was developed within a structured AI agent workflow system. The following terms appear throughout the paper.

Agent roles. Each role is an AI agent session with a defined scope of responsibility.

Table 1: Agent roles referenced in this paper.

Role	Responsibility
Researcher	Designs and executes experimental studies
Protocol Tester (PT)	Performs temporal, entry-by-entry analysis; identifies operational improvements and behavioral-contract candidates
Technowizard (TW)	Performs structural synthesis; ranks improvements by leverage and attributes overhead sources
Facilitating agent	Prepares and manages the study environment for each execution run (this role was coined during the analyzed trace)
Gardener	Manages dependency analysis and structural-change risk assessment
Synthesist	Performs evidence-weighting and cross-document pattern matching

Document types. Agents work within a structured document system.

Table 2: Document types referenced in this paper.

Type	Purpose
Brief	A task specification with acceptance criteria that scopes a unit of work
Inquest	A structured investigation of an operational defect, following a defined sprint protocol
Journal	An analytical record organized as sequential entries, each documenting observations from a defined scope
Account	A post-execution record documenting what happened against the brief’s acceptance criteria
Form	A structured template that defines required fields and checkpoints for a document type
TRACE	A per-session provenance record documenting what the agent did and when
Campaign	A coordination document that tracks bundled work items and priority constraints

System concepts.

- **Form gate:** a required checkpoint in a form that must pass before the agent can proceed to the next step.
- **Taboo:** a behavioral contract that prevents locally rational but globally harmful actions. Identified by the X/Y/Z structure: X is the rational behavior, Y is the structural condition that permits it, Z is the globally bad outcome.
- **Thinking trace:** the extended planning text an AI agent produces before and during task execution. This text is not preserved in the agent’s output documents. It is recoverable only from the model’s reasoning output.

2 The Two-Pass Structure and Why Each Pass Is Necessary

2.1 Pass 1: Piece-by-Piece (Protocol Tester)

The first pass moves through the thinking trace in temporal order, treating each reasoning segment as a unit of analysis. For each segment, it extracts:

- **Operational improvements:** what the agent discovered reactively that a form gate should have surfaced proactively.
- **Taboo opportunities:** structural decision points where locally rational behavior could produce a globally bad outcome. These are candidates for behavioral contracts.
- **Vocabulary coinages:** terms the agent invents when the form has no prompt for the concept the agent is working with.
- **What worked well:** efficient behaviors the agent self-directed correctly, which the form does not require but should.

The piece-by-piece pass preserves temporal sequence. This matters because the same constraint (a slug collision check, for example) may be identified in entry 1 as a concern, again in entry 2 as a confirmed gap, and resolved in entry 3, but at the wrong stage. The temporal record shows whether the agent caught the constraint early or late, and whether the form prompted the check or the agent generated it independently.

Vocabulary coinages are a diagnostic signal specific to this pass. When an agent coins a term mid-execution (“vector-papering,” “facilitating agent,” “annotation-resolution parity,” “gap-fire phase,” “designer’s hypothesis map”), the coinage marks a structural gap. The agent is inventing language because the form has no field for the concept. Vocabulary that is not coined but used fluently (from role files or prior forms) does not carry this signal. Tracking coinages across the trace identifies where the form’s conceptual coverage ends.

2.2 Pass 2: Structural Synthesis (Technowizard)

The second pass reads the whole trace for system-level patterns. It does not re-analyze individual segments. It builds on the piece-by-piece findings but uses them as evidence for structural claims rather than as its primary output.

The structural pass extracts:

- **Ranked system improvements:** what form or protocol changes would reduce overhead across all future executions of this type, ranked by leverage (how many sessions the fix improves and by how much).
- **Overhead source attribution:** for each improvement, whether the overhead source is the system (form lacks a gate) or the agent (judgment pattern worth correcting).
- **Specific agent advice:** per-behavior guidance concrete enough to change behavior on the next run. “Do X before Y,” not “be more careful.”
- **Efficiency baselines:** what the agent did correctly that should not be disturbed. Examples: parallel reads at session start, format-calibration reads before writing, targeted verification before committing.

The structural pass also performs a correction function. Because it reads the full trace after the piece-by-piece pass, it can identify where individual segment analysis overclaimed. In the ouija-inquest-design trace, the piece-by-piece pass raised a concern about a near-collision

between two hypotheses that both turned on timing. For the intermediate scenario design (hypotheses DEFAULT-UNSCOPED, TRIGGERS-MISSING-ENTRY, PROTOCOL-INTENTIONAL-SPLIT), the structural pass pushed back: those hypotheses turn on different evidence types, so the near-collision concern was partly misplaced there. That scenario was later abandoned when the protocol files it referenced turned out not to exist. The concern did apply, however, to the final scenario design (the INDEX.md stale active entry; hypotheses SEQUENCE-STALE, BRIEF-RECLASSIFIED, PARALLEL-WORK): SEQUENCE-STALE and PARALLEL-WORK both turn on timing. There the structural pass named the near-collision but did not resolve it. Segment analysis compresses. The structural pass has context the segment did not, but it does not resolve every concern it inherits.

2.3 Why Neither Pass Is Sufficient Alone

The piece-by-piece pass produces a complete operational trace but no priority ordering. A 30-item improvement list with no leverage ranking is not actionable. The structural pass produces a ranked, attributed list but may miss granular signal (vocabulary coinages, efficiency observations, individual constraint-discovery moments) that does not survive compression into structural claims.

The two passes have complementary coverage maps:

Table 3: Complementary coverage of the two analytical passes.

Piece-by-piece pass captures	Structural pass captures
Temporal sequence of constraint discovery	Priority ordering by leverage
Vocabulary coinages as gap signals	Overhead source attribution
Per-segment “what worked well”	Specific agent advice
Taboo candidate identification	Leverage-ranked system improvements
Individual moment detail	Correction of segment overclaims

A single-pass analysis using either method alone would produce findings, but those findings would be either unprioritized (piece-by-piece alone) or potentially disconnected from granular trace evidence (structural alone).

3 Finding Taxonomy

Four finding types emerged from applying this methodology to the ouija-inquest-design trace. They are described here as a proposed taxonomy. Whether these four types are sufficient or exhaustive across other traces and task types is not yet known (see §6.5).

3.1 Operational Improvements

Specific form or process changes that would prevent a class of overhead. Distinguished from agent advice by their addressability: an operational improvement changes what a form requires; agent advice changes what an agent does when the form is silent.

An operational improvement is correctly identified when the agent did the right thing self-directed, without form prompting, and when the same behavior could be made mechanical via a gate. The analysis guide structure (six components: diagnostic reach, diagnostic assumption, paper-over, dead-end naming criterion, gap-fire phase, baseline adjacency confirmation) is an operational improvement. The Researcher invented the entire structure mid-execution. A future facilitating agent reading only the form would have no prompt for any of it. Making the analysis guide a mandatory form section with required fields converts form-design work into field-population work. That is a straightforward leverage gain.

3.2 Taboo Opportunities

Structural decision points where the agent is locally rational but where a missing gate creates conditions for a globally bad outcome. Identified by the X/Y/Z structure: X (locally rational behavior), Y (structural condition that permits it), Z (globally bad outcome).

Taboo opportunities from thinking trace analysis are candidates, not registered taboos. They route to a triage queue for the Protocol Tester, not directly to the registry. The thinking trace is the evidence base for the candidate. The piece-by-piece pass records where in the trace the candidate was identified.

From the ouija-inquest-design trace, three distinct taboo opportunities emerged across the six entries:

- **experimental-inquest-scenario-design**: covers the pre-design gate and analysis guide.
- **experimental-inquest-run-facilitation**: covers the facilitating agent role definition and run setup/teardown.
- **experimental-inquest-analysis-guide**: covers the analysis guide’s mandatory fields specifically, if not bundled with scenario-design.

3.3 Agent Behavior Signals

Patterns in the agent’s approach that no form change would address, because they are judgment calls: sequencing decisions, awareness of when design and execution are blurring, meta-habits about form submission timing.

These are identified when the agent’s approach has a structural signature across the trace. A one-time mistake is not a behavior signal. A recurrent pattern is. The dissolution of the design/execution boundary (the agent designing planted inquest content and simultaneously running its investigation sprint in its head) occurs in Blocks 3, 4, and 5. The Campaign-read ordering error (Campaign state read at close, not at open) occurs twice. A three-occurrence pattern is a behavioral signal worth specific advice.

Agent behavior signals are addressable only via advice, not via form changes, because they represent agent initiative applied in the wrong order, not a missing form prompt.

3.4 Efficiency Patterns

What the agent did well that should not be disturbed. These are identified explicitly because the default tendency of improvement-focused analysis is to produce a list of deficits without recording the baseline. Without efficiency patterns, a remediation pass has no way to distinguish “this is fine, don’t touch it” from “this is also a problem.”

From the ouija-inquest-design trace, the efficiency patterns were: five parallel reads at session open (correct bootstrapping), two format-calibration reads before writing any artifact (produces authentic-looking output), directory verification calls before committing files, the “neither artifact is wrong” insight applied to the artifact before archival (not just recorded in the account), and the analysis guide’s four-field annotation scheme (the highest-value output of the execution despite being form-unguided).

4 Findings from the Ouija-Inquest-Design Trace

4.1 The Dominant Overhead Pattern: Reactive Constraint Discovery

The Researcher made the right call on every major constraint: artifact existence, slug collision, immutability, path verification, baseline adjacency, analysis guide structure. None of these were

errors. Every one was discovered reactively, as it arose in the execution. The EXPERIMENTAL_INQUEST_FORM handed the Researcher a blank canvas when it should have handed them a constraint checklist.

The cost is visible as reasoning cycles burned on scenarios that were always going to fail. Multiple scenario designs were built and abandoned across the trace. Two collapsed when the protocol files they referenced turned out not to exist; others were abandoned after slug collision checks, internal consistency failures, or newly discovered constraints. A single directory-existence check at the start (an `ls` of the protocols scaffolding directory) could have killed the first two in 30 seconds. All overhead before the final scenario (the INDEX.md stale active entry) is attributable to the absence of a pre-design constraint gate.

This is the highest-leverage system improvement: a mandatory pre-design checklist (file existence, slug collision, immutability constraint, artifact path declaration, hypothesis independence check) that converts reactive discovery into a front-loaded gate.

4.2 The Analysis Guide as the Most Important Missing Form Section

The analysis guide’s six components were invented by the Researcher in reasoning and do not appear as required fields anywhere in the EXPERIMENTAL_INQUEST_FORM. The six components are: diagnostic reach vs. standard hypothesis formation, diagnostic assumption vs. standard working assumption, diagnostic paper-over vs. appropriate hedging, dead-end naming criterion, gap-fire phase with calibrated round estimate, and baseline adjacency confirmation.

This is the most consequential form gap in the trace. The analysis guide is the primary output used by the facilitating agent during execution runs. A facilitating agent reading only the form has no prompt to include any of these components. The Researcher’s work in this execution is the template. It should become the form’s required structure.

The gap-fire phase and self-correction path assessment are particularly important omissions. Gap-fire phase specifies when in the investigation cycle the diagnostic reach is most likely to fire. For the ouija trap, that is Round 2 or later; early Round 1 reaches are less diagnostically useful. Self-correction path assessment specifies whether the executing agent can correct a diagnostic reach using available artifacts alone. For the ouija trap, the answer is no: the required information is not in any artifact. Without these two fields, an analysis guide is underspecified as a measurement tool.

4.3 The Git Provenance Crack

The planted inquest is dated 2026-03-22 but was committed 2026-03-23. The analysis guide explicitly recommends `git log` as a standard investigative tool for establishing timestamp sequences. An executing agent who follows this recommendation will see that the planted inquest was created after its stated date. This directly contradicts the TRACE-omission justification (“predates the mandatory TRACE check”).

This is the single largest authenticity crack in the design. It was not flagged anywhere in the artifacts and is not acknowledged in the analysis guide. The form should require a git provenance assessment step: after artifacts are designed, before they are written, state whether the planted artifact’s commit date will contradict its stated date, and if so, either note this in the analysis guide or address it before execution.

4.4 The Facilitating Agent Role

The facilitating agent was invented entirely within the artifact. This role creates the TRACE document before each run, copies the EXPERIMENTAL_INQUEST to the active directory, annotates observation fields after each run, and manages planted artifact staging and teardown. The role is load-bearing. Without it, the EXPERIMENTAL_INQUEST does not run. It is

currently defined only inside one artifact. That makes it undiscoverable unless that artifact is read, and unreliable unless every Researcher who designs an EXPERIMENTAL_INQUEST invents the same role in the same way.

The TRACE creation responsibility, the run-numbering convention (`run[N]`), and the Campaign state timing (check at run start, not post-artifact) all require a canonical protocol file, not an inference from one execution’s reasoning blocks.

4.5 Vocabulary Coinages as Gap Signals

Five terms were coined mid-execution where the form had no prompt for the concept:

- **Vector-papering**: treating reconstructed sequence as established intent. Papering over the gap between “we can know the sequence” and “we can know what the completing agent intended.” Named at the point the Researcher was articulating what the trap measures.
- **Facilitating agent**: the role that prepares the study environment before a run. Coined when the path conflict forced the Researcher to invent the role on the spot.
- **Annotation-resolution parity**: the baseline run resolved all annotation types; divergence in the experimental run on the same annotation types is the diagnostic signal. The sharpest statement of what the measurement tool is measuring.
- **Gap-fire phase**: the calibrated point in the investigation cycle where the diagnostic reach is most likely to occur. Named because the form had no field for it.
- **Designer’s hypothesis map**: the intended hypothesis set, documented for the facilitating agent, with collapse-prone pairs flagged. Named when the H1/H3 near-collision concern surfaced and the analysis guide had no way to record it.

Each coinage marks the boundary of the form’s conceptual coverage. A vocabulary coinage is not a failure. It is evidence that the Researcher reached something real that the form has not yet formalized.

5 System Overhead vs. Agent Overhead

5.1 System-Generated Overhead

System overhead arises when the agent performs the right behavior self-directed, without form prompting. The fix is always a form or protocol change: adding the required behavior as a gate. System overhead would be eliminated for all future executions, regardless of the individual agent.

5.2 Agent-Generated Overhead

Agent overhead arises when the agent’s own approach has a recurrent pattern that the form cannot prevent, because it involves sequencing and judgment. The fix is specific behavior advice. It changes what the agent does, not what the form requires.

Design/execution boundary dissolution. The Researcher was writing planted artifact content and simultaneously running the investigation sprint in its head, across three separate blocks. This produces artifacts whose content is partly designed and partly discovered mid-write. That is a harder-to-track boundary than a committed draft followed by a verification pass. The advice: write a complete design draft, then run the verification sprint, then commit the file. The sprint is a check, not a co-author.

Table 4: System-generated overhead in the ouija-inquest-design trace.

Overhead	Fix type	What changes
Pre-design constraint discovery (two abandoned scenarios)	Pre-design gate	Form gains mandatory pre-design checklist
Analysis guide structure invented from scratch	Mandatory form section	Analysis guide fields (six components) become required
Artifact list grew from 3 to 7 items across execution	Artifact enumeration step	Complete artifact list locked before first file is written
Facilitating agent role invented in artifact	Canonical protocol file	Protocol file for experimental-inquest facilitation
Scenario pivot had no re-entry protocol	Pivot re-entry gate	If any gate fails, full design gate restarts
Campaign state read at close, not open	Pre-execution checklist	Campaign read moved to first step of execution
Verification step self-generated	Closure checklist	Form closure step: verify all deliverables exist at stated paths

Form update deferral. The analysis guide structure (the highest-value output of the execution) remains institutional knowledge in reasoning blocks that are not committed anywhere load-bearing. The Researcher identified this at close and did not open a brief for the form update before archiving. The advice: when you invent structure that should be a form requirement, open the brief before closing the session, not as a deferred to-do.

“Neither artifact is wrong” applied at close time. The Researcher identified at account-writing time that the strongest trap design ensures no artifact looks obviously broken. Both are defensible individually; the inconsistency only appears when the artifacts are read together against a background question. This is a trap design heuristic, not a close-time observation. The advice: apply it at design time, after locking the scenario. Ask: “is there an artifact in this set that looks obviously wrong?” Redesign until there is not.

5.3 Why the Distinction Matters

Applying agent advice to system-generated overhead produces brittle outcomes. If a future agent has a different attention pattern, the reactive constraint discovery returns. Applying form changes to agent-generated overhead is category confusion. Forms cannot require judgment-sequencing.

The system/agent attribution is the analytical output that justifies the two-pass structure. The piece-by-piece pass identifies the overhead. The structural pass determines its source and the appropriate remediation type.

6 Limitations

6.1 Absence Is Invisible

Thinking-trace analysis can identify what the agent did, when the agent did it, and whether the form prompted it. It cannot identify what the agent did not think. If a constraint was never noticed (not discovered reactively, not self-corrected, not mentioned in passing), the trace offers no signal. The git provenance crack was flagged by both analysts because it was visible in the artifact (the analysis guide recommends `git log`). An analogous crack that produced no artifact trace would be invisible to both passes.

This is not a fixable limitation. It means that thinking-trace analysis is reliable for the overhead it observes, and silent on the overhead it does not.

6.2 Segmentation Is Analyst-Dependent

The piece-by-piece pass requires the analyst to divide the thinking trace into segments. The PT analysis used six blocks aligned to natural phase breaks: setup through “need to see archived structure”; artifact design through first draft; artifact drafting through directory verification; artifact writing through sprint execution; artifact writing through EXPERIMENTAL_INQUEST completion; account through close. A different analyst might segment more finely (one segment per design decision) or more coarsely (design phase / execution phase / close phase). The choice affects what the temporal sequence shows. A finer segmentation surfaces more moment-level signals. A coarser segmentation risks compressing away the reactive-discovery timing that matters for system overhead attribution.

No specification currently exists for how to segment a thinking trace. This is an open methodological question.

6.3 Vocabulary Coinages Require Naming to Be Visible

The five vocabulary coinages identified in §4.5 were visible because the Researcher explicitly named them. Implicit vocabulary (concepts that shaped behavior without being lexically marked) is not visible in the trace. An agent who consistently treats artifact lists as complete when they are partial, without ever naming the assumption, would not show a vocabulary gap. It would show a recurring scope-expansion pattern that looks behavioral rather than conceptual.

6.4 “What Worked Well” Is Structurally Underweighted

The analytical attention in both passes naturally concentrates on gaps. Efficiency patterns (§3.4) are identified, but as a secondary category. Over multiple trace analyses, this asymmetry could produce a picture of agent behavior that is systematically negative: every trace reduced to a list of what the agent got wrong. The efficiency baseline matters because it is the thing to protect. It is the set of correct behaviors that form changes might inadvertently discourage by adding overhead around them.

6.5 Single-Trace Calibration

The taxonomy and methodology in this study derive from one trace, one agent (the Researcher), and one task type (experimental inquest design). The six reasoning blocks in this trace map to five phases (setup, design, artifact, execution, close). Whether that five-phase structure is intrinsic to agent traces or specific to inquest design work is not determinable from one instance. Whether the system/agent overhead distinction holds at similar proportions for other role types or task types is unknown.

7 Open Research

7.1 Overlap with Constructed-Environment Observation

The ouija trap methodology (the companion study listed as source document 5 in Table 5) and thinking-trace analysis are two instruments for observing agent behavior. The ouija trap observes an executing agent navigating a structural gap during genuine task execution. The agent has no meta-knowledge of the observation. Thinking-trace analysis observes the planning and design agent explicitly reasoning about what it is doing.

These are different behavioral surfaces. The ouija trap cannot see the Researcher’s design process. Thinking-trace analysis cannot see what the executing agent does in the moment of encountering the gap during a sealed run. They are likely to surface non-overlapping findings. The ouija trap reveals how executing agents respond to a gap in their environment. Thinking-trace analysis reveals how designing agents build gaps into the environment they create.

Whether they ever surface overlapping findings (the same behavioral pattern visible from both directions) is an empirical question. A confirmed overlap would be strong evidence for a stable behavioral pattern, not an artifact of one observation method.

7.2 Methodology Generalization to Other Roles

The piece-by-piece + structural methodology was developed for a Researcher trace. The Researcher’s task (experimental inquest design) produces extended reasoning about form constraints, artifact design, and scenario calibration. It is a rich trace for operational improvement extraction.

Different roles would produce different trace structures. A Protocol Tester trace would be rich in taboo candidate reasoning and behavioral contract scoping. A Gardener trace would be rich in dependency analysis and structural-change risk assessment. A Synthesist trace would be rich in evidence-weighting and pattern-matching across source documents. Whether the same four finding types (operational improvements, taboo opportunities, agent behavior signals, efficiency patterns) appear across all role traces is unknown. The two-pass structure (temporal extraction + structural synthesis) should generalize, because the overhead source attribution question (system vs. agent) applies to any role’s reasoning.

7.3 Vocabulary Coinage as a Systematic Form-Gap Detector

If vocabulary coinages reliably mark the boundaries of form conceptual coverage, then tracking coinages across a corpus of thinking traces would produce a map of where the form suite’s coverage ends. This could be useful for form maintenance: rather than waiting for a form gap to produce a visible overhead pattern, coinage detection in traces could surface latent gaps before they accumulate cost.

This depends on the reliability of the coinage signal. Some coinages are productive. Annotation-resolution parity and designer’s hypothesis map both became required form fields. Some coinages mark invented infrastructure that was needed for one execution and may not generalize (the specific facilitating agent invocation pattern, for example). Distinguishing generalizable coinages from one-off inventions may require multiple traces.

7.4 Thinking-Trace Analysis as a Standing Practice

This methodology was applied once to one trace. Its value scales with frequency. The ouija-inquest-design trace covered six blocks and produced three taboo candidates, five vocabulary coinages, seven system improvements (§5.1, consolidated from both passes), and three agent behavior signals, all from a single task execution. A standing practice of trace analysis after significant design executions would compound: form gaps identified in one session become form requirements that reduce overhead in the next.

The question is resourcing. Trace analysis as practiced here required two full analytical passes (PT and TW), each treating the trace at depth. This is not trivially cheap. Whether a lighter-weight variant (a single-pass structural read, or a PT pass only) would capture sufficient signal to justify the reduction in rigor is worth testing.

7.5 Source Documents and Data Availability

Source documents 1 through 4 are deposited alongside this paper as supplementary files. They are also committed in the repository at `papers/thinking-trace-analysis/sources/`.

Table 5: Source documents for this study.

#	Filename	Contribution
1	JOURNAL_ouija-researcher-analysis_2026-03-23.md	Six-entry temporal pass: operational improvements, taboo candidates, vocabulary, and “what worked well” per segment
2	JOURNAL_technowizard-researcher-analysis_2026-03-23.md	Whole-trace structural read: six system improvements ranked by leverage, seven agent behavior advice items
3	BRIEF_ouija-inquest-design_2026-03-22.md	The task specification the Researcher executed. The thinking trace was the model’s reasoning output during this execution; it is not stored in the brief
4	STUDY_STUB_thinking-trace-analysis_2026-03-26.md	Scoping document that identified this as a study, not an account, and proposed the section structure
5	Ouija methodology study	Available from the author on request. Companion methodology study

A Source Document Summaries

Brief descriptions of each source document follow. The full texts of documents 1 through 4 are co-deposited as supplementary files.

A.1 PT Analysis (Piece-by-Piece)

The Protocol Tester journal (297 lines) walks through the Researcher’s thinking trace in six entries aligned to natural phase breaks. Each entry extracts four categories: operational improvements, taboo opportunities, new vocabulary, and what worked well. The journal closes with a whole-thing synthesis that rolls up the per-entry findings into three taboo candidates and advice to the Researcher on the git provenance crack, the H1/H3 near-collision, and the “too easy” calibration concern.

A.2 TW Analysis (Structural)

The Technowizard journal (432 lines) reads the same trace for system-level patterns. It produces two ranked outputs: six system improvements ordered by leverage (analysis guide structure, pre-design gate, artifact enumeration, facilitating agent protocol, git provenance step, campaign read timing) and seven items of specific agent advice (existence checks first, complete artifact list, separate design from execution, read campaign first, submit form updates before closing, apply the “neither artifact is wrong” heuristic at design time, note the git provenance crack).

A.3 Design Brief

The design brief (157 lines) is the task specification for the ouija-inquest-design execution. It defines the problem statement, acceptance criteria, and context the Researcher worked from. The thinking trace itself was the model’s reasoning output during this execution. It is not stored in the brief file. The PT and TW journals (source documents 1 and 2) quote the trace

extensively across their analyses. The trace spans six reasoning blocks covering scenario design, artifact drafting, sprint execution, and task closure.

A.4 Study Stub

The study stub (64 lines) is the scoping document that identified the two analytical journals as constituting a methodology rather than a record. It proposed the section structure used in this paper and named the research question: what does extended thinking text reveal about agent planning and design behavior, and can systematic multi-pass analysis reliably surface operational improvements and structural taboo candidates?